

SQLD 58회 기출 특강

유형1. 다양한 COUNT 문제

- WHERE 조건에 맞는 행이 없는 경우 COUNT 결과는 0이다.
- WHERE 조건에 맞는 행이 없는 경우 SUM, AVG, MIN, MAX 결과는 NULL이다.
- HAVING 조건에 맞는 그룹이 없는 경우 COUNT, SUM, AVG, MIN, MAX 결과는 공집합이다.

문제1. 다음 두 SQL 실행 결과를 순서대로 작성한 것은?

<TAB1>	
COL1	
ABCD	
ACDE	
BCDE	
CDEA	

1)

```
SELECT COUNT(*)  
  FROM TAB1  
 WHERE COL1 LIKE 'a%';
```

2)

```
SELECT COUNT(*)  
  FROM TAB1  
 HAVING COUNT(*) > 4;
```

- ① 0, 0
- ② 0, 공집합
- ③ NULL, 0
- ④ NULL, 공집합

문제2. 다음 SQL문의 실행 결과로 알맞은 것은?

<TAB1>

COL1	COL2	COL3
A	X	100
A	X	200
B	X	200
B	Y	300

<TAB2>

COL4	COL5
100	e
200	E
300	a

```
SELECT COUNT(*)
FROM (SELECT DISTINCT COL1, COL2
      FROM TAB1
      WHERE COL3 IN (SELECT COL4
                     FROM TAB2
                     WHERE UPPER(COL5) = 'E')) X;
```

- ① 1
- ② 2
- ③ 3
- ④ 4

문제3. 다음 SQL 문의 실행 결과를 순서대로 나열한 것은?

<TAB1>

COL1	COL2	COL3
10	NULL	30
NULL	30	0
0	40	NULL

```
SELECT AVG(COL3) FROM TAB1;
SELECT AVG(COL3) FROM TAB1 WHERE COL1 > 0;
SELECT AVG(COL3) FROM TAB1 WHERE COL1 IS NOT NULL;
```

- ① 10, 15, 15
- ② 10, 30, 30
- ③ 15, 15, 15
- ④ 15, 30, 30

유형2. WITH문 (공통 테이블 식, CTE: Common Table Expression)

- WITH문은 임시로 이름이 붙은 결과집합(가상 테이블)을 정의하는 구문이다.
- 한 번 정의한 쿼리 결과를 여러 번 사용할 수 있어 가독성과 유지보수성이 높아진다.
- 복잡한 서브쿼리를 WITH절로 분리하여 쿼리 구조를 단순화할 때 자주 사용된다.
- 한 번 정의된 CTE는 그 아래의 메인 쿼리(SELECT, INSERT, UPDATE, DELETE) 내에서만 사용 가능하다.

[문법]

```
WITH 별칭1 AS (SELECT ...),  
     별칭2 AS (SELECT ...)  
SELECT ...  
  FROM 별칭1, 별칭2  
WHERE ... ;
```

문제1. 다음 쿼리의 실행 결과로 알 수 있는 내용을 고르시오.

```
WITH 학생성적평균 AS (  
  SELECT 학년, AVG(성적) AS 평균성적  
  FROM 학생  
  GROUP BY 학년  
)  
SELECT S.이름, S.학년, S.성적  
  FROM 학생 S JOIN 학생성적평균 G ON S.학년= G.학년  
 WHERE S.성적> G.평균성적;
```

- ① 모든 학생의 평균 점수를 구하는 쿼리이다.
- ② 학년별 평균 점수보다 높은 학생만 출력한다.
- ③ 전체 평균보다 높은 학생만 출력한다.
- ④ 학년별 최고 점수를 가진 학생만 출력한다.

문제2. 아래 SQL 설명으로 적절한 것은?

선수	<u>선수ID</u> , 선수명, 나이, 급여, 소속
팀	<u>팀ID</u> , 팀명
소속	<u>선수ID</u> , <u>팀ID</u>

※ 굵게 표시된 컬럼이 각 테이블의 기본키이다.

```
WITH X AS (SELECT T.팀ID, T.팀명, SUM(P.급여) AS 총급여
            FROM 팀 T JOIN 소속 M ON T.팀ID = M.팀ID
            JOIN 선수 P ON M.선수ID = P.선수ID
            GROUP BY T.팀ID, T.팀명)
SELECT X.팀명
       FROM X, (SELECT MAX(총급여) AS 총급여
                FROM X) Y
WHERE X.총급여 = Y.총급여;
```

- ① 각 팀의 평균 급여를 계산한다.
- ② 팀별로 가장 급여가 높은 선수를 구한다.
- ③ 모든 팀의 총급여 합계를 구한다.
- ④ 선수들의 총급여가 가장 높은 팀명을 반환한다.

유형3. ESCAPE CHARACTER

- ESCAPE CHARACTER는 원래는 '특수한 의미(와일드카드·정규식 메타문자)'로 사용되는 문자를, 그 문자 그대로(문자 자체) 사용하고자 할 때 지정하는 해제용 문자를 의미한다.

1) LIKE에서 ESCAPE 사용법

- LIKE는 %, _ 가 패턴용 와일드카드
- 그래서 이 문자 자체를 검색하려면 ESCAPE 키워드로 직접 선언해야 함

```
SELECT *  
  FROM TAB1  
 WHERE COL1 LIKE '%/_%' ESCAPE '/';
```

2) 정규식(REGEXP_LIKE 등)에서 ESCAPE 사용법

- 정규식에서는 \ (역슬래시) 가 기본적인 escape 방식
- [] 안에 들어가는 특수기호도 일반기호처럼 그대로 해석됨
- LIKE의 ESCAPE 키워드 ❌ 사용 안 함

```
SELECT *  
  FROM TAB1  
 WHERE REGEXP_LIKE(COL1, '[^]');  
  
SELECT *  
  FROM TAB1  
 WHERE REGEXP_LIKE(COL1, '\^');
```

문제1. 아래 SQL을 만족하는 행의 수는?

<TAB1>	
COL1	
AC	
ABD	
ADC	
ABCD	
A_C	
BA_CD	

```
SELECT *
  FROM TAB1
 WHERE COL1 LIKE '%A_C%'
 UNION ALL
SELECT *
  FROM TAB1
 WHERE COL1 LIKE '%A\C%' ESCAPE '\';
```

- ① 0
- ② 2
- ③ 4
- ④ 6

문제2. 아래 SQL을 만족하는 행의 수는?

<TAB1>	
COL1	
A1234	
AA1234	
A+1234	
AA+1234	
A1234+	

```
SELECT COUNT(*)
  FROM TAB1
 WHERE REGEXP_LIKE(COL1, 'A[+]');
```

- ① 0
- ② 2
- ③ 4
- ④ 5

유형4. 계층형 질의(LEVEL)

- LEVEL 은 1부터 시작하며, CONNECT BY 조건이 참일 동안 계속 1씩 증가합니다.
- START WITH를 생략하면 루트(시작점)가 없기 때문에 전 테이블의 모든 행이 루트 노드가 됩니다.

문제1. 다음 SQL 실행 결과는?

```
SELECT COUNT(*)  
  FROM DUAL  
CONNECT BY LEVEL <= 2;
```

- ① 공집합
- ② 0
- ③ NULL
- ④ 2

문제2. 다음 SQL 실행 결과는?

```
SELECT SUM(LEVEL)  
  FROM DUAL  
CONNECT BY LEVEL <= 3;
```

- ① 0
- ② 1
- ③ 3
- ④ 6

문제3. 다음 SQL 실행 결과는?

<DEPT>		
DEPTNO	DNAME	PARENT_DEPT
10	총무부	NULL
20	인사부	10
30	재무부	20

```
SELECT COUNT(*)  
  FROM DEPT  
CONNECT BY PRIOR DEPTNO = PARENT_DEPT;
```

- ① 0
- ② 1
- ③ 3
- ④ 6

유형5. 기타 유형

- SQL에서는 문자열을 표현할 때 '(단일 인용부호, Single Quote)를 사용
- 문자열 내부에서 ' 자체를 표현하려면 '' (작은따옴표 2개)로 표현함

문제1. 다음 SQL 실행결과로 옳은 것은?

```
SELECT '''A''''  
FROM DUAL;
```

- ① 에러발생
- ② A
- ③ 'A''
- ④ 'A'''